

Defining Recursive Functions on Lists

Lecture Notes Transcription

Course: Interactive Theorem Proving (7CCSACTP)

1. Context & List Representation Intuition

Last Week Revision:

- Introduction to Isabelle interface (terms, theorems)
- Proof styles: FW, BW, induction, substitution, cases

This Video Agenda:

- Defining recursive functions on lists (Theory + Isabelle)

List Intuition & Structure:

- Any list xs is constructed via:
 - `Nil` (`[]`)
 - `Cons x xs` ($x\#xs$)
- Syntactic sugar:

$$a\#[] \approx [a], \quad [a, b, c] \approx a\#b\#c\#[]$$

- Uses: Arrays, Association lists (e.g. `[(Bob, 23), (Alice, 34)]`)
- **Finiteness:** Lists in Isabelle are always finite applications of `Cons`: $x_1\#x_2\#\dots\#x_n\#[]$.

2. Recursive Functions on Lists

Concept: A function f on lists is recursive if $f(x\#xs)$ is evaluated in terms of $f(xs)$ (calling itself on the tail). A fully-defined recursive function must cover all input patterns (`[]` and $x\#xs$).

Example 1: Sum of List

- **Signature:** `sum :: nat list  $\Rightarrow$  nat`
- **Equations:**
 - `sum [] = 0`
 - `sum (x # xs) = x + sum xs`
- **Evaluation trace:**

$$\begin{aligned} \text{sum } [0, 1, 3] &= 0 + \text{sum } [1, 3] \\ &= 0 + (1 + \text{sum } [3]) \\ &= 0 + (1 + (3 + \text{sum } [])) \\ &= 0 + (1 + (3 + 0)) = 4 \end{aligned}$$

Example 2: Append

- **Signature:** `append :: 'a list  $\Rightarrow$  'a list  $\Rightarrow$  'a list`
- **Equations:**
 - `append [] ys = ys`
 - `append (x # xs) ys = x # (append xs ys)`
- **Evaluation trace:**

$$\begin{aligned} \text{append } [1, 2] [3] &= 1\#(\text{append } [2] [3]) \\ &= 1\#(2\#(\text{append } [] [3])) \\ &= 1\#(2\#[3]) = [1, 2, 3] \end{aligned}$$

3. Important Rules for Termination & Simpset

- **Termination Requirement:** To guarantee termination, all recursive calls must be on strictly smaller arguments (e.g. xs is structurally smaller than $x\#xs$).
- **Simpset Generation:** Only proven terminating functions will have automatic simplification rules (`f.simps`) generated and added to Isabelle's simpset.